

PiRouter

First-Time Setup & Deployment Guide

Raspberry Pi + Quectel EC200U LTE + Wi-Fi Hotspot + Web Configuration Portal

Goal

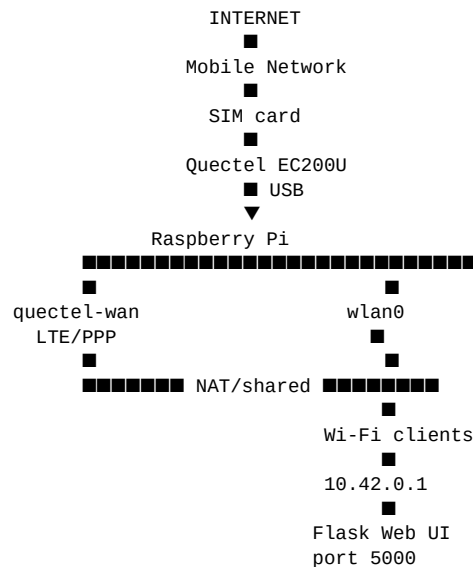
Build a Raspberry Pi that boots as a Wi-Fi router, provides a local configuration webpage, shares cellular Internet through the LTE modem, and lets a user change the Wi-Fi SSID/password and mobile APN without using SSH after initial setup.

Factory behavior: power on → connect to the default hotspot → open **http://10.42.0.1:5000** → configure Wi-Fi and cellular settings → Save & Apply.

Version: PiRouter v1.0 setup baseline

1. What you are building

NetworkManager manages the Wi-Fi access point and cellular connection, ModemManager communicates with the EC200U, and Flask provides the local configuration/status portal.



Critical design rule: Wi-Fi and the configuration portal must remain available even when LTE is disconnected, the SIM is missing, or the APN is wrong.

2. Hardware and prerequisites

- **Raspberry Pi** with Wi-Fi and Raspberry Pi OS.
- **Quectel EC200U** or compatible LTE modem connected by USB.
- SIM card with an active mobile-data plan.
- Stable power supply.
- Laptop/phone for first-time configuration.
- SSH access during development.

APN, username/password, SIM PIN requirements, carrier bands, and provisioning depend on the SIM/carrier.

3. Prepare Raspberry Pi OS

Install Raspberry Pi OS, boot the Pi, and verify SSH access before continuing.

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y network-manager modemmanager python3 python3-flask
```

```
sudo systemctl enable NetworkManager
sudo systemctl enable ModemManager
sudo systemctl start NetworkManager
sudo systemctl start ModemManager
```

4. Connect and verify the LTE modem

```
lsusb
ls -l /dev/ttyUSB*
mmcli -L
mmcli -m 0
```

Useful diagnostics:

```
mmcli -m 0 --output-keyvalue | grep -Ei 'operator|signal|quality|access|technology|state|registration|packet-service'
```

The exact device number and ttyUSB port can vary. Do not hard-code ttyUSB5 without checking the target Pi.

5. Create the Wi-Fi hotspot

Use NetworkManager to create the hotspot. Adjust wlan0 if your Pi uses a different interface.

```
sudo nmcli connection add type wifi ifname wlan0 con-name PiHotspot ssid PiRouter

sudo nmcli connection modify PiHotspot \
  802-11-wireless.mode ap \
  802-11-wireless.band bg \
  802-11-wireless.ssid "PiRouter" \
  wifi-sec.key-mgmt wpa-psk \
  wifi-sec.psk "12345678" \
  ipv4.method shared \
  ipv4.addresses 10.42.0.1/24 \
  connection.autoconnect yes \
  connection.autoconnect-priority 100 \
  connection.autoconnect-retries 0

sudo nmcli connection up PiHotspot

nmcli connection show PiHotspot | grep -E '802-11-wireless.ssid|ipv4.method|ipv4.addresses|connection.autoconnect'
nmcli connection show --active
ip -4 addr show wlan0
```

Expected gateway: **10.42.0.1**. The hotspot must work without Internet.

6. Create the cellular NetworkManager connection

```
sudo nmcli connection add type gsm ifname "" con-name quectel-wan apn "www"

sudo nmcli connection modify quectel-wan \
  gsm.apn "www" \
  ipv4.method auto \
  connection.autoconnect yes \
  connection.autoconnect-priority 50 \
  connection.autoconnect-retries 0

sudo nmcli connection up quectel-wan

nmcli connection show quectel-wan | grep -E 'connection.autoconnect|connection.autoconnect-priority|gsm.apn|GENERAL.STATE|GEN
nmcli connection show --active
ip -4 route
ping -c 3 8.8.8.8
```

7. Create persistent configuration

Runtime settings belong in JSON, not in Python source code. The webpage changes this data; the backend applies it.

```
sudo mkdir -p /etc/pigateway

sudo tee /etc/pigateway/config.json > /dev/null <<'EOF'
{
  "wifi": {
    "ssid": "PiRouter",
    "password": "12345678"
  },
  "modem": {
    "apn": "www",
    "username": "",
    "password": "",
    "pin": ""
  }
}
EOF

sudo chmod 600 /etc/pigateway/config.json
sudo chown root:root /etc/pigateway/config.json
sudo cat /etc/pigateway/config.json
```

The file can contain credentials, so keep it root-owned and mode 600.

8. Create configuration helpers

Create root-owned helpers for reading, validating/writing, and applying configuration. This is safer than giving Flask unrestricted root access.

```
sudo tee /usr/local/sbin/pigateway-config-read > /dev/null <<'EOF'
#!/bin/bash
set -e
CONFIG="/etc/pigateway/config.json"

if [ ! -f "$CONFIG" ]; then
    echo "Configuration file not found" >&2
    exit 1
fi

cat "$CONFIG"
EOF

sudo chmod 750 /usr/local/sbin/pigateway-config-read
sudo chown root:root /usr/local/sbin/pigateway-config-read

sudo -n /usr/local/sbin/pigateway-config-read
```

The project's config-write helper should validate JSON, SSID length, Wi-Fi password length, APN and optional modem credentials, then atomically replace the configuration file.

9. Apply configuration to NetworkManager

The apply helper reads JSON and updates the existing profiles. It should validate first, then apply Wi-Fi and cellular settings.

```
# Core operations inside pigateway-apply:

nmcli connection modify PiHotspot \
    802-11-wireless.ssid "$SSID" \
    wifi-sec.psk "$WIFI_PASSWORD" \
    ipv4.method shared

nmcli connection modify quectel-wan \
    gsm.apn "$APN" \
    gsm.username "$MODEM_USERNAME" \
    gsm.password "$MODEM_PASSWORD"

nmcli connection up quectel-wan
nmcli connection up PiHotspot
```

If an SSID is changed, the current Wi-Fi client will disconnect. That is normal; reconnect to the new SSID.

10. Configure sudo permissions

```
sudo tee /etc/sudoers.d/pigateway > /dev/null <<'EOF'
techiesms ALL=(root) NOPASSWD: /usr/local/sbin/pigateway-config-read
techiesms ALL=(root) NOPASSWD: /usr/local/sbin/pigateway-config-write
techiesms ALL=(root) NOPASSWD: /usr/local/sbin/pigateway-apply
EOF

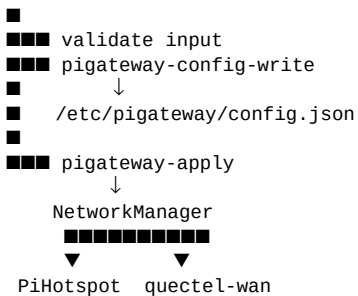
sudo chmod 440 /etc/sudoers.d/pigateway
sudo visudo -cf /etc/sudoers
```

Replace **techiesms** with the actual Linux username on another Pi.

11. Build the Flask web portal

The frontend sends form values to Flask. Flask validates them and invokes the privileged helpers. It does **not** modify app.py.

```
Browser
  ■
  ■ POST /save
  ▼
Flask app.py
```



The portal should expose at least: SSID, Wi-Fi password, APN, APN username, APN password, and SIM PIN (when required), plus Wi-Fi/LTE/Internet status.

Bind Flask to 0.0.0.0:5000 so hotspot clients can reach it.

12. Start the web interface at boot

```
sudo systemctl daemon-reload
sudo systemctl enable gateway.service
sudo systemctl start gateway.service
```

```
sudo systemctl status gateway.service --no-pager
systemctl is-enabled gateway.service
```

The gateway service should use Restart=on-failure so the portal can recover from an application crash.

13. Make boot behavior reliable

NetworkManager should own automatic startup of both profiles:

```
sudo nmcli connection modify PiHotspot \
connection.autoconnect yes \
connection.autoconnect-priority 100 \
connection.autoconnect-retries 0
```

```
sudo nmcli connection modify quetel-wan \
connection.autoconnect yes \
connection.autoconnect-priority 50 \
connection.autoconnect-retries 0
```

Wi-Fi has the higher priority so the portal comes up regardless of LTE success.

If you created a separate `pihotspot.service` solely to run **nmcli connection up PiHotspot**, disable it only after testing NetworkManager autoconnect by itself. Do not disable the Wi-Fi profile.

14. First-time cold-boot test

```
sudo poweroff
```

Remove power completely, wait about 15 seconds, then power the Pi back on.

- Look for the factory SSID **PiRouter**.
- Connect using the factory password.
- Open **http://10.42.0.1:5000**.
- Confirm the portal loads before worrying about LTE.
- Confirm LTE and Internet when a valid SIM/APN is installed.
- Browse the Internet from the connected client.

Success means the device can recover from a real power cycle without SSH or manual network commands.

15. Change Wi-Fi and APN from the webpage

Example Wi-Fi test: change PiRouter to DevRouter and save. Reconnect to DevRouter after the old connection drops.

```
nmcli connection show PiHotspot | grep '802-11-wireless.ssid'
sudo cat /etc/pigateway/config.json
```

Example APN test: change the APN in the webpage, save/apply, then verify the stored and applied values:

```
sudo cat /etc/pigateway/config.json
nmcli connection show quectel-wan | grep 'gsm.apn'
```

A wrong APN for the inserted SIM is a useful negative test: LTE/Internet may fail, but Wi-Fi and the portal should remain accessible. Restore the correct APN afterward.

A second-SIM test is still required to prove multi-carrier behavior. Carrier provisioning, bands, coverage, data plans, and modem compatibility can affect the result.

16. Diagnostics

USB and modem:

```
lsusb
ls -l /dev/ttyUSB*
mmcli -L
mmcli -m 0
```

Network:

```
nmcli device status
nmcli connection show --active
ip -4 addr show wlan0
ip -4 route
```

LTE profile:

```
nmcli connection show quectel-wan | grep -E 'GENERAL.STATE|GENERAL.IP-IFACE|GENERAL.DEFAULT|gsm.apn|IP4.ADDRESS|IP4.GATEWAY|I
```

Internet:

```
ping -c 3 -W 2 8.8.8.8
```

Web service:

```
sudo systemctl status gateway.service --no-pager -l
sudo journalctl -u gateway.service -n 100 --no-pager
curl -i http://127.0.0.1:5000/
curl -i http://10.42.0.1:5000/
```

17. If the webpage shows HTTP 500

HTTP 500 means the request reached Flask but the application failed while processing it. It is different from an unreachable IP.

For example, if `load_config()` calls `json.loads(result.stdout)` and the helper returns empty output, Flask will return 500. Check the helper directly and inspect the service log:

```
sudo -n /usr/local/sbin/pigateway-config-read

python3 - <<'PY'
import subprocess
cmd = ["sudo", "-n", "/usr/local/sbin/pigateway-config-read"]
result = subprocess.run(cmd, capture_output=True, text=True, timeout=10)
print("RETURN CODE:", result.returncode)
print("STDOUT:", repr(result.stdout))
print("STDERR:", repr(result.stderr))
PY

sudo journalctl -u gateway.service -n 100 --no-pager
```

If both localhost and 10.42.0.1 return HTTP 500, the failure is inside Flask/config handling, not the hotspot IP.

18. Recovery and security recommendations

- Keep the configuration portal independent of LTE.
- Validate every submitted value before changing NetworkManager.
- Use atomic configuration writes.
- Keep a known-good factory Wi-Fi configuration.
- Add a physical or software factory-reset mechanism before deployment.
- Do not put personal/production credentials into the master image.
- Restrict sudo to the minimum helper commands.
- Add clear states: Connecting, Connected, Disconnected, Failed.
- Consider authentication for the configuration portal if the device will be deployed outside a trusted environment.

19. Create the master SD-card image

Only image the card after the full cold-boot test succeeds.

- Restore factory defaults: SSID PiRouter, factory password, and blank/placeholder cellular credentials.
- Remove personal data, temporary credentials, logs/history where appropriate, and development artifacts.
- Confirm NetworkManager, ModemManager, and gateway.service are enabled.
- Power the Pi off cleanly.
- Remove the SD card and connect it to your Windows PC.
- Use a disk-imaging tool that can read the complete SD card into an .img file.
- Keep the tested SD card untouched as the golden master.
- Flash copies and perform a cold-boot test on each target.

Suggested filename: **PiRouter-v1.0.img**.

20. Final pre-deployment checklist

Item	Expected result
Boot	Pi boots without manual commands
Wi-Fi	Factory SSID appears automatically
Gateway	10.42.0.1
Web UI	http://10.42.0.1:5000 loads
SSID change	New SSID appears after Save & Apply
Password change	New password works
Persistence	Changes survive reboot/power cycle
USB modem	EC200U detected
LTE	quectel-wan connects with valid SIM/APN
APN change	Web value reaches NetworkManager
Internet	Client browses through LTE
Wrong APN	Wi-Fi/portal remain usable
Second SIM	At least one additional carrier tested

Item	Expected result
Recovery	Factory reset/recovery path documented
Master image	Created only after tests pass

Appendix A. Quick command sheet

```
# USB / modem
lsusb
ls -l /dev/ttyUSB*
mmcli -L
mmcli -m 0

# Wi-Fi
nmcli connection show PiHotspot
nmcli connection show --active
ip -4 addr show wlan0

# LTE
nmcli connection show quectel-wan
ip -4 route

# Internet
ping -c 3 -W 2 8.8.8.8

# Web service
sudo systemctl status gateway.service --no-pager -l
sudo journalctl -u gateway.service -n 100 --no-pager

# Configuration
sudo cat /etc/pigateway/config.json

# Portal tests
curl -i http://127.0.0.1:5000/
curl -i http://10.42.0.1:5000/
```

End state: power on → connect to factory Wi-Fi → open the local portal → enter Wi-Fi and cellular settings → Save & Apply
→ use the Pi as an LTE-to-Wi-Fi router without SSH.